**Lexical analyzer**

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

#define MAX 1000
#define SIZE 100

char *keywords[] = {
   "int", "float", "char", "if", "else",
   "while", "for", "return", "void"
};

int isKeyword(char word[])
{
   int i;

   for (i = 0; i < 9; i++)
   {
      if (strcmp(word, keywords[i]) == 0)
         return 1;
   }

   return 0;
}
```

```
int main()
{
   char input[MAX], token[SIZE];
   int i = 0, j, ch, next;

   printf("Enter a C program (enter END to finish):\n");

   while (fgets(input + i, MAX - i, stdin))
   {
      if (strcmp(input + i, "END\n") == 0 ||
         strcmp(input + i, "END") == 0)
      {
         input[i] = '\0';
         break;
      }

      i += strlen(input + i);

      if (i >= MAX - 1)
         break;
   }

   i = 0;

   printf("\nTOKENS:\n");
   printf("-------\n");
```

```
while ((ch = input[i]) != '\0')
{
   if (isspace(ch))
   {
      i++;
      continue;
   }

   // Single line comment
   if (ch == '/' && input[i + 1] == '/')
   {
      while (input[i] != '\0' && input[i] != '\n')
         i++;

      continue;
   }

   // Multi-line comment
   if (ch == '/' && input[i + 1] == '*')
   {
      i += 2;

      while (input[i] != '\0')
      {
         if (input[i] == '*' && input[i + 1] == '/')
```

```
        {
            i += 2;
            break;
        }

        i++;
    }

    continue;
}

// Keyword or Identifier
if (isalpha(ch) || ch == '_')
{
    j = 0;

    while (isalnum(input[i]) || input[i] == '_')
    {
        if (j < SIZE - 1)
            token[j++] = input[i];

        i++;
    }

    token[j] = '\0';
```

```
    if (isKeyword(token))
        printf("Keyword: %s\n", token);
    else
        printf("Identifier: %s\n", token);

    continue;
}

// Number
if (isdigit(ch))
{
    j = 0;

    while (isdigit(input[i]) || input[i] == '.')
    {
        if (j < SIZE - 1)
            token[j++] = input[i];

        i++;
    }

    token[j] = '\0';

    printf("Number: %s\n", token);
    continue;
}
```

```
// Operators
if (ch == '+' || ch == '-' || ch == '*' ||
   ch == '%' || ch == '=' || ch == '<' ||
   ch == '>')
{
   next = input[i + 1];

   if ((ch == '+' && next == '+') ||
      (ch == '-' && next == '-') ||
      (ch == '=' && next == '=') ||
      (ch == '<' && next == '=') ||
      (ch == '>' && next == '='))
   {
      printf("Operator: %c%c\n", ch, next);
      i += 2;
   }
   else
   {
      printf("Operator: %c\n", ch);
      i++;
   }

   continue;
}
```

```
        // Division operator
        if (ch == '/')
        {
            printf("Operator: /\n");
            i++;
            continue;
        }

        // Symbols
        if (ch == ';' || ch == ',' || ch == '(' ||
            ch == ')' || ch == '{' || ch == '}' ||
            ch == '[' || ch == ']')
        {
            printf("Symbol: %c\n", ch);
            i++;
            continue;
        }

        printf("Unknown: %c\n", ch);
        i++;
    }

    return 0;
}
```

**Lex program**

```
%{
#include <stdio.h>
%}

%%

"int"|"float"|"char"|"if"|"else"|"while"|"return" {
   printf("Keyword: %s\n", yytext);
}

[a-zA-Z_][a-zA-Z0-9_]* {
   printf("Identifier: %s\n", yytext);
}

[0-9]+ {
   printf("Number: %s\n", yytext);
}

"+"|"-"|"*"|"/"|"=" {
   printf("Operator: %s\n", yytext);
}

";"|","|"("|")"|"{"|"}" {
   printf("Special Symbol: %s\n", yytext);
}
```

```
[ \t\n]+ ;

. {
    printf("Unknown: %s\n", yytext);
}

%%

int main()
{
    printf("Enter a C statement:\n");
    yylex();
    return 0;
}

int yywrap()
{
    return 1;
}
```

**Commands:**

### Execution Steps in Red Hat Linux

Step 1: Create a file

```
vi program.l
```

Step 2: Enter the code

- Press `i` to enter Insert mode.
- Type or paste the above code.

Step 3: Save and exit

- Press `Esc`.
- Type `:wq`
- Press `Enter`.

Step 4: Generate the C file

```
lex program.l
```

OR

```
flex program.l
```

Step 5: Compile

For Lex:

```
gcc lex.yy.c -ll
```

For Flex:

```
gcc lex.yy.c -lfl
```

Step 6: Execute



```
./a.out
```

Step 7: Give input

```
int a = 10;
float b = 20;
```

Press `Ctrl + D` to indicate the end of input.

### 3. Example

Input:

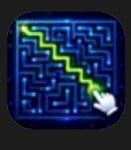
```
int a=10;
```

Output:

```
Keyword: int
Identifier: a
Operator: =
Number: 10
Delimiter: ;
```

## Week - 3

### 1. Algorithm


Arrow Path: Maze Puzzle
AD sponsored
Download

**First and follow**

```
#include <stdio.h>
#include <string.h>

#define MAXP 50
#define MAXNT 30
#define MAXSYM 60
#define MAXR 20
#define LEN 32

typedef struct {
   char lhs[LEN];
   char rhs[MAXR][LEN];
   int len;
} Production;

Production p[MAXP];

char nt[MAXNT][LEN];
char sym[MAXSYM][LEN];

int first[MAXNT][MAXSYM];
int follow[MAXNT][MAXSYM];

int n = 0, nn = 0, ns = 0;
```

```
int getNT(char *s) {
   for (int i = 0; i < nn; i++)
      if (strcmp(nt[i], s) == 0)
         return i;
   return -1;
}

void addNT(char *s) {
   if (getNT(s) == -1)
      strcpy(nt[nn++], s);
}

int getSym(char *s) {
   for (int i = 0; i < ns; i++)
      if (strcmp(sym[i], s) == 0)
         return i;

   strcpy(sym[ns], s);
   return ns++;
}

int add(int set[], int x) {
   if (!set[x]) {
      set[x] = 1;
      return 1;
   }
```

```
    return 0;
}

void saveProduction(char *lhs, char rhs[][LEN],
                    int len) {
    strcpy(p[n].lhs, lhs);
    p[n].len = len;

    for (int i = 0; i < len; i++)
        strcpy(p[n].rhs[i], rhs[i]);

    n++;
}

int main() {
    int i, j, k, changed;

    char line[200];

    printf("Enter number of grammar rules: ");
    scanf("%d", &n);

    int rules = n;
    n = 0;

    getchar();
```

```
printf("Enter grammar (use spaces around -> and |):\n");

for (i = 0; i < rules; i++) {
    char lhs[LEN];
    char rhs[MAXR][LEN];
    int len = 0;

    fgets(line, sizeof(line), stdin);

    char *token = strtok(line, " \t\r\n");

    if (token == NULL)
        continue;

    strcpy(lhs, token);
    addNT(lhs);

    token = strtok(NULL, " \t\r\n"); // ->

    while ((token = strtok(NULL, " \t\r\n")) != NULL) {

        if (strcmp(token, "|") == 0) {
            saveProduction(lhs, rhs, len);
            len = 0;
        }
```

```
        else {
            strcpy(rhs[len++], token);
        }
    }

    saveProduction(lhs, rhs, len);
}

int epsilon = getSym("#");
int dollar = getSym("$");

// FIRST calculation

do {
    changed = 0;

    for (i = 0; i < n; i++) {
        int A = getNT(p[i].lhs);
        int nullable = 1;

        if (strcmp(p[i].rhs[0], "#") == 0) {
            changed |= add(first[A], epsilon);
            continue;
        }

        for (j = 0; j < p[i].len; j++) {
```

```
            int B = getNT(p[i].rhs[j]);

            if (B == -1) {
               int x = getSym(p[i].rhs[j]);
               changed |= add(first[A], x);
               nullable = 0;
               break;
            }

            for (k = 0; k < ns; k++) {
               if (k != epsilon && first[B][k])
                  changed |= add(first[A], k);
            }

            if (!first[B][epsilon]) {
               nullable = 0;
               break;
            }
         }

         if (nullable)
            changed |= add(first[A], epsilon);
      }

   } while (changed);
```

```
// FOLLOW calculation

add(follow[0], dollar);

do {
   changed = 0;

   for (i = 0; i < n; i++) {
      int A = getNT(p[i].lhs);

      for (j = 0; j < p[i].len; j++) {
         int B = getNT(p[i].rhs[j]);

         if (B == -1)
            continue;

         int nullable = 1;

         for (k = j + 1; k < p[i].len; k++) {
            int C = getNT(p[i].rhs[k]);

            if (C == -1) {
               int x = getSym(p[i].rhs[k]);
               changed |= add(follow[B], x);
               nullable = 0;
               break;
```

```
            }

            for (int t = 0; t < ns; t++) {
                if (t != epsilon && first[C][t])
                    changed |= add(follow[B], t);
            }

            if (!first[C][epsilon]) {
                nullable = 0;
                break;
            }
        }

        if (nullable) {
            for (k = 0; k < ns; k++) {
                if (follow[A][k])
                    changed |= add(follow[B], k);
            }
        }
    }
  }

} while (changed);

// Print FIRST sets
```

```
printf("\nFIRST SETS:\n");

for (i = 0; i < nn; i++) {
   printf("FIRST(%s) = { ", nt[i]);

   for (j = 0; j < ns; j++) {
      if (first[i][j])
         printf("%s ", sym[j]);
   }

   printf("}\n");
}

// Print FOLLOW sets

printf("\nFOLLOW SETS:\n");

for (i = 0; i < nn; i++) {
   printf("FOLLOW(%s) = { ", nt[i]);

   for (j = 0; j < ns; j++) {
      if (follow[i][j])
         printf("%s ", sym[j]);
   }

   printf("}\n");
```

```
    }

    return 0;
}
```

**Predictive parser**

```
#include <stdio.h>
#include <string.h>

char stack[100];
int top = -1;

void push(char c) {
    stack[++top] = c;
}

char pop() {
    return stack[top--];
}

void parse(char s[]) {
    int i = 0;

    push('$');
    push('E');
```

```
while (1) {
    char X = stack[top];
    char a = s[i];

    if (X == '$' && a == '$') {
        printf("Accepted");
        return;
    }

    if (X == a) {
        pop();
        i++;
        continue;
    }

    if (X == 'E' && (a == 'i' || a == '(')) {
        pop();
        push('A');
        push('T');
    }
    else if (X == 'A' && a == '+') {
        pop();
        push('A');
        push('T');
        push('+');
```

```
}
else if (X == 'A' && (a == ')' || a == '$')) {
    pop();
}
else if (X == 'T' && (a == 'i' || a == '(')) {
    pop();
    push('B');
    push('F');
}
else if (X == 'B' && a == '*') {
    pop();
    push('B');
    push('F');
    push('*');
}
else if (X == 'B' && (a == '+' || a == ')' || a == '$')) {
    pop();
}
else if (X == 'F' && a == 'i') {
    pop();
    push('i');
}
else if (X == 'F' && a == '(') {
    pop();
    push(')');
    push('E');
```

```
            push('(');
        }
        else {
            printf("Error");
            return;
        }
    }
}

int main() {
    char input[100];

    printf("Enter string ending with $ (use i for id): ");
    scanf("%99s", input);

    parse(input);

    return 0;
}
```

**LALR program**

```
#include <stdio.h>
#include <ctype.h>

int action[7][3] = {
```

```
    {4, 5, 0},
    {0, 0, 999},
    {4, 5, 0},
    {4, 5, 0},
    {-3, -3, -3},
    {0, 0, -1},
    {-2, -2, -2}
};

int goTo[7][2] = {
    {1, 2},
    {-1, -1},
    {-1, 5},
    {-1, 6},
    {-1, -1},
    {-1, -1},
    {-1, -1}
};

char terminals[] = {'c', 'd', '$'};
char nonTerminals[] = {'S', 'C'};

char lhs[] = {' ', 'S', 'C', 'C'};
int len[] = {0, 2, 2, 1};

char *production[] = {
```

```
    "",
    "S -> CC",
    "C -> cC",
    "C -> d"
};

int getCol(char ch)
{
    int i;
    for(i = 0; i < 3; i++)
        if(terminals[i] == ch)
            return i;
    return -1;
}

int getGotoCol(char ch)
{
    int i;
    for(i = 0; i < 2; i++)
        if(nonTerminals[i] == ch)
            return i;
    return -1;
}

int main()
{
```

```
char input[100], str[100];
int stack[100];
int i, j = 0;
int top = 0, pos = 0;
int state, col, act, p, g;

printf("Enter input string: ");
fgets(str, sizeof(str), stdin);

for(i = 0; str[i] != '\0'; i++)
   if(!isspace((unsigned char)str[i]))
      input[j++] = str[i];

input[j++] = '$';
input[j] = '\0';

stack[0] = 0;

printf("\nINPUT: %s\n", input);
printf("\nSTACK\tINPUT\tACTION\n");

while(1)
{
   state = stack[top];
   col = getCol(input[pos]);
```

```
if(col == -1)
{
   printf("ERROR\nSTRING REJECTED\n");
   break;
}

act = action[state][col];

for(i = 0; i <= top; i++)
   printf("%d ", stack[i]);

printf("\t%s\t", input + pos);

if(act == 999)
{
   printf("ACCEPT\nSTRING ACCEPTED\n");
   break;
}
else if(act == 0)
{
   printf("ERROR\nSTRING REJECTED\n");
   break;
}
else if(act > 0)
{
   printf("SHIFT %c\n", input[pos]);
```

```
        stack[++top] = act - 1;
        pos++;
    }
    else
    {
        p = -act;

        printf("REDUCE: %s\n", production[p]);

        top = top - len[p];

        g = getGotoCol(lhs[p]);

        state = goTo[stack[top]][g];

        if(state == -1)
        {
            printf("STRING REJECTED\n");
            break;
        }

        stack[++top] = state;
    }
}

return 0;
```

```
}
```

**Three address code**

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

char expr[100];
int pos = 0;
int temp = 1;

void expression(char result[]);
void term(char result[]);
void factor(char result[]);

void newTemp(char result[])
{
   sprintf(result, "t%d", temp++);
}

void factor(char result[])
{
   if (expr[pos] == '(')
   {
      pos++;
```

```
        expression(result);

        if (expr[pos] == ')')
            pos++;
        else
            strcpy(result, "ERROR");

        return;
    }

    if (isalnum((unsigned char)expr[pos]))
    {
        int i = 0;

        while (isalnum((unsigned char)expr[pos]))
            result[i++] = expr[pos++];

        result[i] = '\0';
    }
    else
    {
        strcpy(result, "ERROR");
    }
}

void term(char result[])
```

```
{
    char left[20], right[20], t[20];
    char op;

    factor(left);

    while (expr[pos] == '*' || expr[pos] == '/')
    {
        op = expr[pos++];
        factor(right);

        if (strcmp(left, "ERROR") == 0 ||
            strcmp(right, "ERROR") == 0)
        {
            strcpy(result, "ERROR");
            return;
        }

        newTemp(t);

        printf("%s = %s %c %s\n", t, left, op, right);

        strcpy(left, t);
    }

    strcpy(result, left);
```

```
}

void expression(char result[])
{
   char left[20], right[20], t[20];
   char op;

   term(left);

   while (expr[pos] == '+' || expr[pos] == '-')
   {
      op = expr[pos++];
      term(right);

      if (strcmp(left, "ERROR") == 0 ||
         strcmp(right, "ERROR") == 0)
      {
         strcpy(result, "ERROR");
         return;
      }

      newTemp(t);

      printf("%s = %s %c %s\n", t, left, op, right);

      strcpy(left, t);
```

```
    }

    strcpy(result, left);
}

int main()
{
    char lhs[20], result[20];

    printf("Enter expression (example: a=b+c*d): ");
    scanf("%99s", expr);

    char *p = strchr(expr, '=');

    if (p != NULL)
    {
        int len = p - expr;

        if (len == 0 || len >= 20)
        {
            printf("Invalid expression\n");
            return 0;
        }

        strncpy(lhs, expr, len);
        lhs[len] = '\0';
```

```
    for (int i = 0; lhs[i] != '\0'; i++)
    {
        if (!isalnum((unsigned char)lhs[i]))
        {
            printf("Invalid expression\n");
            return 0;
        }
    }

    pos = (int)(p - expr) + 1;
}
else
{
    strcpy(lhs, "result");
    pos = 0;
}

printf("\nThree Address Code:\n");

expression(result);

if (strcmp(result, "ERROR") == 0 || expr[pos] != '\0')
{
    printf("Invalid expression\n");
    return 0;
```

```
    }

    printf("%s = %s\n", lhs, result);

    return 0;
}
```

**AST program**

```
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>

struct Node
{
    char data;
    struct Node *left, *right;
};

char expr[100];
int pos = 0;
int reg = 0;

struct Node* createNode(char c)
{
    struct Node *n = (struct Node*)malloc(sizeof(struct Node));
```

```
    n->data = c;
    n->left = NULL;
    n->right = NULL;

    return n;
}

struct Node* factor();
struct Node* term();
struct Node* expression();

struct Node* factor()
{
    if (isalnum(expr[pos]))
        return createNode(expr[pos++]);

    if (expr[pos] == '(')
    {
        pos++;

        struct Node *n = expression();

        if (expr[pos] == ')')
            pos++;
        else
            return NULL;
```

```
        return n;
    }

    return NULL;
}

struct Node* term()
{
    struct Node *n = factor();

    if (n == NULL)
        return NULL;

    while (expr[pos] == '*' || expr[pos] == '/')
    {
        char op = expr[pos++];

        struct Node *r = factor();

        if (r == NULL)
            return NULL;

        struct Node *p = createNode(op);
        p->left = n;
        p->right = r;
```

```
        n = p;
    }

    return n;
}

struct Node* expression()
{
    struct Node *n = term();

    if (n == NULL)
        return NULL;

    while (expr[pos] == '+' || expr[pos] == '-')
    {
        char op = expr[pos++];

        struct Node *r = term();

        if (r == NULL)
            return NULL;

        struct Node *p = createNode(op);
        p->left = n;
        p->right = r;
        n = p;
```

```
    }

    return n;
}

int generate(struct Node *n)
{
    if (n == NULL)
        return -1;

    if (n->left == NULL && n->right == NULL)
    {
        int r = reg++;
        printf("LOAD R%d, %c\n", r, n->data);
        return r;
    }

    int r1 = generate(n->left);
    int r2 = generate(n->right);

    if (r1 == -1 || r2 == -1)
        return -1;

    switch (n->data)
    {
        case '+': printf("ADD R%d, R%d\n", r1, r2); break;
```

```
        case '-': printf("SUB R%d, R%d\n", r1, r2); break;
        case '*': printf("MUL R%d, R%d\n", r1, r2); break;
        case '/': printf("DIV R%d, R%d\n", r1, r2); break;
    }

    return r1;
}

int main()
{
    printf("Enter expression: ");
    scanf("%99s", expr);

    struct Node *root = expression();

    if (root == NULL || expr[pos] != '\0')
    {
        printf("Invalid expression\n");
        return 0;
    }

    printf("\nGenerated Machine Code:\n");

    int r = generate(root);

    if (r != -1)
```

```
        printf("STORE result, R%d\n", r);


    return 0;
}
```